

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 03 -

CONTROLE DE VERSÕES DE DEPENDÊNCIAS (CÓDIGO, API, MODELO, DADOS)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Imagine essa cena, que se repete com frequência em equipes de ciência de dados: um modelo foi treinado, validado, documentado e entregue. Semanas depois, alguém tenta reproduzir os resultados — seja um colega, um revisor, ou o próprio autor retornando ao projeto, ou até mesmo a publicação do modelo em um ambiente produtivo — e nada funciona. Os números são diferentes. Um pacote não instala. O script levanta um erro que não existia antes. O ambiente que um dia funcionou virou um quebra-cabeça sem solução óbvia.

Esse cenário não é resultado de descuido ou falta de profissionalismo. É a consequência natural de um ecossistema que, por muito tempo, não tratou reprodutibilidade como um requisito de primeira classe.

Entender este problema, e como lidar com ele, é importante porque é a base da resolução dos problemas de reprodutibilidade de ambiente. Se já vimos nas aulas anteriores que este problema existe, agora é hora de focar com mais cuidado e atenção nele.

HANDS ON

Nos vídeos, vamos abordar algumas práticas importantes para ajudar no controle dos diferentes componentes da nossa solução e ver como usar algumas ferramentas extras para ajudar a manter os ambientes reproduzíveis e funcionais em qualquer ambiente.

Para além disso, vamos discutir em detalhes os principais problemas que enfrentamos quando precisamos gerenciar as dependências. Entender estes problemas, que são como peças móveis extremamente sensíveis em uma engrenagem complexa, é a diferença entre sucesso e fracasso no processo de produtização de modelos.

SAIBA MAIS

O Python é o problema?

Vamos do princípio, para entender a questão da linguagem e seu ecossistema: o Python nasceu e cresceu de forma orgânica, com ferramentas sendo criadas por comunidades diferentes para resolver problemas específicos, sem uma visão unificada de como um projeto deveria ser gerenciado do início ao fim.

Até porque por muito tempo não se considerava a possibilidade de projetos complexos construídos em Python e a quantidade gigantesca de bibliotecas que surgiria, fazendo com que um ecossistema emergisse advindo de um caos que parecia organizado, dependendo de onde se olhasse para a solução de distribuição escolhida.

O resultado foi uma cadeia de dependências frágil, onde qualquer elo, qualquer componente, seja a versão do interpretador, a versão de um pacote, o sistema operacional, as bibliotecas nativas do sistema, pode se quebrar silenciosamente.

Outro ponto importante: o Python é, por essência, uma linguagem interpretada e mesmo com algumas melhorias de desempenho, a estrutura é mantida: não compilamos um programa para distribuição e sim transportamos uma quantidade de arquivos que constituem o software. Com isso, o funcionamento não depende apenas do ambiente operacional onde o código vai rodar, mas também do runtime e bibliotecas que são dependentes do ambiente operacional onde o mesmo código vai rodar.

E para ciência de dados, esse problema é ainda mais crítico do que em desenvolvimento de software tradicional. Um bug em uma aplicação rest/web costuma ser simples de identificar: a funcionalidade para de funcionar, o erro aparece no retorno dos dados. Mas um bug de reprodutibilidade em um modelo pode ser invisível: o código roda até o fim, produz resultados, mas os resultados são sutilmente diferentes. Uma versão ligeiramente diferente do numpy pode mudar o comportamento de operações de álgebra linear. Uma versão diferente do scikit-learn pode alterar o critério de desempate em algoritmos de árvore. O modelo "funciona",

mas não é o mesmo modelo e o processo não foi alterado, não existe um erro por se.

Por isso, o problema a se endereçar é muito mais estrutural: precisamos que o ambiente seja o mesmo, do ponto de vista da aplicação. E quando falamos “o mesmo”, queremos dizer que tem que ser as mesmas bibliotecas, nas mesmas versões e que essas bibliotecas tenham o mesmo comportamento. E isso é uma tarefa que pode ser mais complicada ainda, dependendo da complexidade do trabalho feito e das dependências específicas.

Vamos olhar para cada uma dessas complexidades de forma mais aprofundada.

O interpretador

Como citado anteriormente, o Python em si é uma variável da complexidade. Diferente de linguagens com versões mais estáveis e retrocompatíveis, o Python evolui quebrando compatibilidade de forma intencional e planejada. Cada evolução da linguagem, por mais que prometa algum nível de retrocompatibilidade, introduz mudanças na linguagem, como as mais recentes versões, onde houve mudanças importantes no comportamento de tipos nativos, na biblioteca padrão e no sistema de tipagem, que afetam o funcionamento do código.

E daí temos um problema que acontece usualmente, mesmo que hoje em dia um pouco menos, de que a maioria dos ambientes de desenvolvimento não controla explicitamente a versão do interpretador. A pessoa começa a trabalhar com o Python “que está na máquina” e quando esse código, criado para uma versão do Python, precisa rodar em outro ambiente, com outra versão do Python, o comportamento de todo o código que depende de características específicas da versão pode mudar e muitas vezes de forma silenciosa.

E em ciência de dados, isso é um complicador relevante, porque a maioria das bibliotecas de ML e computação científica tem dependências nativas, código compilado em C, C++ ou Fortran que precisa ser recompilado para cada versão do interpretador. O numpy, o scipy, o pandas, o torch e praticamente toda biblioteca de alto desempenho distribuem “wheels” (versões especiais de pacotes) pré-

compiladas para combinações específicas de versão do Python e sistema operacional. Quando a versão do Python não tem uma wheel disponível, o instalador tenta compilar o pacote a partir do código-fonte — e aí o ambiente precisa ter compiladores, headers de sistema e bibliotecas nativas instaladas, o que raramente está garantido em máquinas de desenvolvimento ou servidores de CI.

Uma nova versão do Python? Já vou atualizar... ou será que não?

O Python tem um ciclo de lançamento anual. Uma nova versão minor (3.12, 3.13, ...) é lançada usualmente em outubro de cada ano, trazendo melhorias de performance, novos recursos de linguagem e, ocasionalmente, remoções de funcionalidades depreciadas. Mas o ecossistema científico não acompanha esse ritmo com a mesma cadência.

Bibliotecas como numpy, scipy e pandas são projetos complexos, com extenso código nativo em C e Fortran, mantidos por equipes relativamente pequenas de voluntários. Suportar uma nova versão do Python exige recompilar extensões nativas, atualizar bindings, testar comportamentos e publicar wheels pré-compiladas para todas as plataformas suportadas. Esse processo leva tempo — tipicamente alguns meses após o lançamento do Python.

Então temos que nos primeiros meses após o lançamento de uma nova versão do Python, usar essa versão em projetos significa não ter as bibliotecas preparadas e para isso existem três saídas: aguardar a atualização das bibliotecas; compilar tudo a partir do código-fonte (o que exige um ambiente de build configurado e funcional); ou encontrar erros de instalação e de uso sem mensagens claras. Projetos que dependem de bibliotecas menos populares ou menos mantidas podem esperar muito mais — ou descobrir que a biblioteca simplesmente nunca será atualizada.

Esses comportamentos não são problemas das bibliotecas, nem do Python. São consequências inevitáveis de um ecossistema em movimento, onde a compatibilidade retroativa tem limites. A lição prática é clara: a versão mais recente do Python não é necessariamente a melhor escolha para um projeto de ciência de dados. A escolha da versão deve ser feita com base na compatibilidade com as bibliotecas do projeto, não no desejo de usar o runtime mais novo.

Dependências e dependências que dependem de dependências...

Mesmo usando soluções mais modernas, como falamos em aulas anteriores, ainda assim quando um projeto declara dependências, raramente declara apenas as diretas. Pandas depende de numpy. scikit-learn depende de numpy e scipy. matplotlib depende de numpy, pillow e contourpy. Cada um desses pacotes tem suas próprias dependências e assim por diante. Um projeto de ciência de dados modesto pode facilmente ter 10 dependências diretas e 80 ou 100 dependências transitivas (ou seja, dependências de uma dependência principal).

E daí temos o conhecido “Inferno das Versões” (Dependency Hell), problema que surge quando esses pacotes declaram restrições de versão incompatíveis entre si e daí o processo de instalação falha, ou pior ainda, uma atualização de um pacote transitivo quebra algo que o pacote direto assumia. Um exemplo perto da nossa realidade: uma biblioteca de visualização assume que numpy expõe uma certa API interna. O numpy lança uma versão que remove essa API. A biblioteca de visualização ainda não foi atualizada. O ambiente quebra — mas o erro não aponta para a causa raiz de forma clara.

Sem um mecanismo que registre as versões exatas de todas as dependências (diretas e transitivas), duas instalações ou sync de ambiente em momentos diferentes podem produzir ambientes completamente diferentes, mesmo usando o requirements.txt ou pyproject.toml definido no projeto. Isso acontece porque, se não registrada a versão corretamente, o instalador vai instalar a versão mais recente compatível com a restrição declarada — e “mais recente” muda com o tempo.

E é importante, novamente, lembrar um ponto: existe uma percepção perigosa no desenvolvimento de software analítico de que atualizar a versão do interpretador Python (por exemplo, migrar do Python 3.9 para o 3.12) resultará apenas em ganhos de performance e novas sintaxes, sem impactos colaterais. Mas, como falamos há pouco de forma clara, a ciência de dados é diretamente apoiada em bibliotecas que dependem de integrações profundas com a linguagem e, conseqüentemente, com o runtime.

E ainda existe o ambiente de operação...

Se a versão do Python é uma variável, o sistema operacional e a arquitetura do equipamento onde o processamento deve ser feito é outra, frequentemente mais traiçoeira, porque seus efeitos são menos óbvios e mais difíceis de diagnosticar.

A raiz do problema está nos pacotes com extensões nativas. Quando um pacote como numpy ou lightgbm é instalado, o que está sendo instalado não é apenas código Python puro — é uma combinação de código Python e código compilado (geralmente em C, C++ ou Fortran) que interage diretamente com o sistema operacional, sistema operacional este que é adaptado para a arquitetura do equipamento. Esse código compilado precisa ser construído para a arquitetura e o sistema operacional alvo e o comportamento pode diferir entre plataformas de formas sutis.

Temos dois exemplos interessantes:

- O processador Apple Silicon: quando as pessoas que trabalham em projetos de dados começaram a usar a linha de processadores Apple Silicon, não haviam wheels para esta arquitetura e nem os anteriores funcionavam adequadamente, porque a arquitetura mudou completamente. Então por algum tempo haviam duas soluções: rodar as bibliotecas com a camada de emulação que a Apple fornecia, e suportar a degradação da performance e comportamentos diferentes em cálculos complexos, ou esperar que a biblioteca ou wheel estivesse disponível. E ainda assim, outras bibliotecas demoraram muito tempo para fazer uma versão compatível e outras até hoje ainda têm uma versão específica para esta arquitetura.
- Windows e bibliotecas nativas: diferente do Linux e macOS, o Windows não vem com compiladores de C/C++ instalados por padrão. O ecossistema de ferramentas de compilação no Windows é bastante fragmentado, cada uma com suas particularidades (há o Visual Studio Build Tools, o MinGW, o MSYS2) e cada um tem suas peculiaridades de compatibilidade. Para pacotes que precisam ser compilados a partir do código-fonte (seja porque não há wheel disponível para a versão do Python, seja porque o pacote não distribui wheels para Windows), o

processo de instalação no Windows frequentemente falha com erros relacionados a compiladores ausentes, variáveis de ambiente não configuradas ou headers de sistema não encontrados. Isso faz com que se dependa de wheels compilados por pessoas da comunidade, e ainda assim não é possível garantir o funcionamento semelhante em todos os ambientes.

E como resolver?

Temos um conjunto de boas práticas, que pode ser adotado em todos os projetos, em maior ou menor escala. É responsabilidade do time que cria o projeto preparar uma estrutura de reprodutibilidade adequada e aqui podemos indicar algumas ideias.

A declaração explícita da versão do Python é uma premissa a definir desde o primeiro momento. Ferramentas como uv (com `uv python pin`) e pyenv (com `.python-version`) garantem que todos os colaboradores e ambientes de CI usem exatamente a mesma versão do interpretador — não "Python 3.11 ou superior", mas "Python 3.11.7". Essa especificidade elimina toda uma classe de incompatibilidades.

O uso de lockfiles ajuda no processo de validação da instalação dos pacotes. Seja o `uv.lock`, o `poetry.lock` ou o `pip-tools` com `requirements.txt` “pinado” com hashes, o lockfile garante que a resolução de dependências — um processo que pode produzir resultados diferentes em momentos diferentes — seja feita uma única vez e seu resultado versionado junto ao código. Qualquer ambiente criado a partir do lockfile é, por definição, idêntico ao original.

A separação entre dependências de projeto e dependências de desenvolvimento é uma prática frequentemente ignorada, mas que tem impacto direto na reprodutibilidade. `pytest`, `ruff`, `mypy` e ferramentas similares não deveriam fazer parte do ambiente de produção ou inferência – são pacotes usados para apoio no processo de criação de projetos e sua presença aumenta a superfície de possíveis conflitos sem agregar valor ao ambiente de execução. O próprio jupyter tem que ser avaliado, se realmente precisa estar disponível em um ambiente de operação para uso geral externo.

E, por fim, mas muito importante, a documentação dos requisitos de sistema, como as versões de CUDA para projetos com GPU, bibliotecas nativas necessárias, versão do sistema operacional testada. Essa documentação cobre a parte de configuração que os lockfiles Python não conseguem preencher. Essa documentação pode ser até executável: um Dockerfile, um script de setup ou um arquivo de configuração de ambiente que possa ser reproduzido por qualquer membro da equipe sem necessidade de interpretação.

Sempre importante lembrar: reprodutibilidade não é um estado que se atinge uma vez e está resolvido em si. É uma característica de projeto que precisa ser mantida ativamente à medida que o projeto evolui, as dependências são atualizadas e o time cresce. As ferramentas modernas tornam esse trabalho significativamente mais simples — mas apenas se forem usadas de forma consistente e consciente.

O QUE VOCÊ VIU NESTA AULA?

Entendemos que o ecossistema Python, que cresceu sem uma visão unificada de gerenciamento de projetos, resultou inicialmente em projetos com uma cadeia de dependências frágil onde qualquer componente, seja a versão do interpretador, pacotes ou sistema operacional pode causar problemas, que podem ser identificados rapidamente ou corroer a estrutura e o funcionamento do modelo. Em ciência de dados isso é especialmente perigoso: o código pode rodar sem erros e mesmo assim produzir resultados diferentes, porque uma versão ligeiramente diferente do numpy ou do scikit-learn já é suficiente para mudar o comportamento de um modelo.

Para lidar com isso, algumas práticas básicas precisam ser adotadas em todo projeto: declarar explicitamente a versão do Python, usar lockfiles para garantir que as mesmas versões de pacotes sejam instaladas em qualquer máquina, separar dependências de desenvolvimento das dependências reais do projeto e documentar os requisitos de sistema que vão além do Python — como versões de CUDA ou bibliotecas nativas. Reprodutibilidade não é algo que se resolve uma vez: é uma característica que precisa ser mantida ativamente ao longo de todo o projeto.

REFERÊNCIAS

BLANCAS, E. **Setting up reproducible Python environments for Data Science**. Disponível em: <https://ploomber.io/blog/python-envs/>; Acesso em: 23 abr. 2026.02 abril 2026.

HUYEN, C. **Designing Machine Learning Systems**. [s.l.]: O'Reilly Media, 2022.

MAHAMULKAR, P. **Machine Learning Operations (MLOps) For Beginners**. 2024. Disponível em: <https://towardsdatascience.com/machine-learning-operations-mlops-for-beginners-a5686bfe02b2/>. Acesso em: 21 abr. 2026.

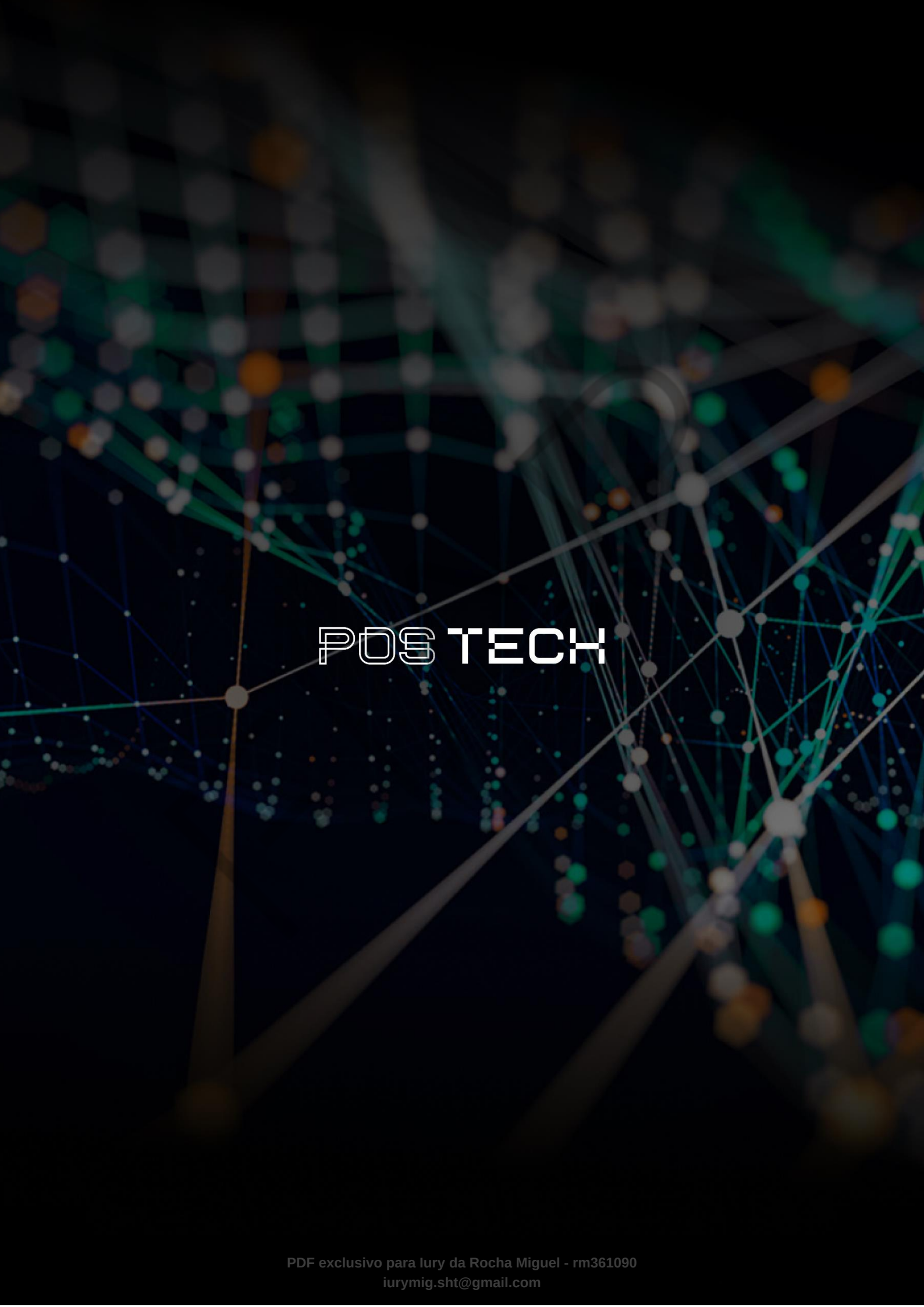
MLOPS. **Machine Learning Operations**. 2026. Disponível em: <https://ml-ops.org/>. Acesso em: 21 abr. 2026.

POTE, S. **Machine Learning in Production**. [s.l.]: BPB Publications, 2023.

PALAVRAS-CHAVE

Compatibilidade. Ambiente de execução. Reprodutibilidade.

EMSE



POSTECH